Book Track

Book Scraper Dashboard

Ali Hassan
SHAH TECH.

Contents

Overview 2
Architecture 2
Technology Stack 2
Project Structure 3
Database Schema 4
Web Scraping System 6
REST API 6
Frontend Dashboard 7
Filters 7
Deployment on Railway 8
Getting Started 8
Running the Application 9
Configuration 9
Troubleshooting 10
Support 10

Overview

Book-Scraper-Dashboard is a full-stack web application that scrapes book listings from multiple online marketplaces and provides an interactive dashboard for browsing, filtering, and analyzing book market data.

Key Features

- Multi-marketplace web scraper (Vinted, Wallapop)
- Real-time analytics dashboard
- Advanced filtering and search capabilities
- Interest tracking (save favorites, mark as not interested)
- Price trends and market statistics
- User authentication
- Responsive web interface

Architecture

Technology Stack

Backend

- Framework: Django 5.1
- API: Django REST Framework
- Database: PostgreSQL
- Web Scraping: Scrapy

Frontend

- HTML / CSS / JavaScript (Vanilla JS)
- Responsive Design using CSS Grid and Flexbox

Deployment

- WSGI / ASGI compatible
- Docker-ready configuration

Project Structure

```
Book-Scraper-Dashboard/
backend/
  api/
    models.py
    views.py
    serializers.py
    services.py
    urls.py

  books_scraper/
    main.py
    search_lists/
    spiders/
      database.py
      models.py
      vinted.py
      wallapop.py

  templates/
    dashboard.html
    login.html
```

```
  static/
    css/
    js/
      dashboard.js
      renderResults.js
      statistics.js
      analytics.js
      price-validation.js
      utilities.js

  settings.py
  urls.py
  wsgi.py
  asgi.py
  manage.py
requirements.txt
```

Database Schema

The system uses three core models:

Source

Represents a marketplace configuration.

Fields:

- spider_id
- spider_name
- spider_domain

History

Tracks ISBN availability per marketplace.

Fields:

• history_id

• site_id

• isbn

• available_books

• sold_books

Detail

Represents individual book listings.

Fields:

• detail_id

• history_id

• isbn

• name

• price

• seller

• condition

• editorial

• images

• url

• site_id

• date_scraped // updated scraped date

• first_seen // first scraped date

• availability

• interest

• contact

Web Scraping System

The scraper system uses a SequentialRunner that runs spiders sequentially.

Workflow:

1. Read search terms from text files.

2. Create spider jobs using search terms.

3. Run spiders sequentially.

4. Save results to the database.

Supported Spiders:

• VintedSpider – Scrapes listings from Vinted marketplace.

• WallapopSpider – Uses Wallapop JSON API to collect listings.

REST API

Authentication

• /auth/login/

• /auth/logout/

Endpoints

Marketplace Names

GET /api/marketplace_names/

Listings

GET /api/details/

Statistics

GET /api/main_stats/

Interest Update

PATCH /api/interest/<detail_id>/

Contact Update

PATCH /api/contact/<detail_id>/

Frontend Dashboard

The dashboard interface has three panels:

Dashboard

Shows real-time statistics and marketplace overview.

Books Explorer

Allows users to browse and filter book listings.(In future)

Analytics

Displays advanced analytics such as price trends and seller insights.

Filters

Available filters include:

- Marketplace selection
- Price range

- Book condition
- Listing age
- Interest status
- Contact status

Deployment on Railway

1. Create Django Service

   Go to settings tab of this service:
   Link your project repo with Django service
   Put custom command section: python install -r requirements.txt
   Put start command: python manage.py 0.0.0.0:8000.
   In variables tab add required shared variables

2. Create Postgres Service

   Connect local app and run migrations and create it superuser.

3. Create Scraping Service

   Link with same repo like Django service
   Put start command: python books_scraper/main.py
   And set schedule

Note: Before running app just ensure you locally connect your hosting db service and run migration and then create this db superuser.

Getting Started

Installation Steps

1. Clone repository

git clone https://github.com/username/Book-Scraper-Dashboard.git

2. Create virtual environment

python -m venv venv

3. Install dependencies

pip install -r requirements.txt

4. Configure PostgreSQL database

5. Run migrations

python manage.py migrate

6. Create admin user

python manage.py createsuperuser

Running the Application

Start Django server:

python manage.py runserver

Run Scrapers:

python books_scraper/main.py

Configuration

Database configuration is defined in Django settings.py.

The system uses PostgreSQL as the primary database.

Scrapy configuration includes:

- User agent rotation
- Download delay
- Cookie handling

Troubleshooting

Common Issues

Scraper Not Finding Items

- Verify search terms
- Check ISBN format
- Confirm marketplace availability

Database Errors

- Ensure PostgreSQL is running
- Verify credentials in settings.py
- Run migrations

Frontend Issues

- Clear browser cache
- Check browser console for errors
- Verify API endpoints

Support

For support:

- Open a GitHub issue
- Review project documentation

- Check code comments for implementation details